Université Abdelmalek Essaâdi
Faculté des Sciences de Tétouan

كلية العلوم ـ تطوان
ⵜⴰⵖⵣⵓⴰⵜ ⵏ ⵜⵓⵙⵙⴰⵏⵉ - ⵜⵣⵓⴰⵡ
Faculté des Sciences - Tétouan

Licence – semestre 5

Ingénierie Informatique et Développement Logicie (2IDL)

Modélisation statistique et analyse de données

Chapitre 3

web scraping

Année Universitaire : 2026/2027

Prof. Ouissam ZEALOUK

PLAN

- Introduction au Web Scraping
- Manipulation des Pages HTML avec urllib.request
- Selenium pour le Web Scraping Dynamique
- Web Scraping avec les APIs
- Exercices

❑ Definition

- Le Web scraping est une méthode automatisée pour extraire des informations de sites internet de manière rapide et structurée. Cette technique remplace le processus de copie manuelle en utilisant un programme pour accéder aux pages web, identifier les données d'intérêt, puis les collecter dans un format structuré (comme CSV, JSON ou une base de données).

Exemples de données couramment collectées :

- Informations de contact (numéros de téléphone, e-mails)
- Informations sur les produits (prix, images, descriptions)
- Avis clients et autres données d'analyse

Utilisateurs et applications :

- Particuliers : pour comparer les prix, suivre les promotions
- Entreprises : pour la veille concurrentielle, les études de marché, l'analyse de tendances et la prise de décisions éclairées

❑ Étapes Clés du Web Scraping

1. Définir les Objectifs et les Sources

- On identifie les données spécifiques dont on a besoin, par exemple, les prix des produits, les avis clients ou des informations de contact.
- On sélectionne ensuite les sites ou pages à scraper en fonction de ces besoins, en s'assurant que la collecte respecte les politiques d'utilisation des sites.

2. Choisir un Outil de Web Scraping

- Outils sans code : Certains logiciels offrent des interfaces graphiques permettant de configurer le scraping sans connaissances en programmation.
- Programmation avec Python : Une option flexible et puissante, avec des bibliothèques comme BeautifulSoup (pour les pages statiques) et Selenium (pour les pages dynamiques).

❑ Étapes Clés du Web Scraping

3. Extraire les Contenus des Pages

- Une fois l'outil choisi, on envoie des requêtes aux URLs ciblées. L'outil analyse la structure HTML de la page et repère les éléments contenant les données souhaitées.
- On identifie ensuite les balises HTML pertinentes (comme , <h2>, <a>) pour extraire les informations.

4. Stocker les Données

- On organise les données collectées dans un format structuré (fichiers CSV, JSON, bases de données) pour une analyse ou un traitement ultérieur.

❑ Applications et Avantages du Web Scraping

Automatisation de la Collecte de Données

- Le Web scraping permet de récupérer automatiquement de grandes quantités de données provenant de multiples sources en ligne, de façon bien plus rapide et efficace qu'une saisie manuelle.
- Cette automatisation réduit le temps et les efforts nécessaires, tout en minimisant les erreurs et en optimisant les ressources.

Surveillance des Prix et Veille Concurrentielle

- Le Web scraping est un outil puissant pour surveiller l'évolution des prix dans des secteurs compétitifs, permettant aux entreprises d'ajuster leurs stratégies tarifaires en conséquence.
- Pour les consommateurs, cela facilite la comparaison des prix et aide à trouver les meilleures offres disponibles.

❑ Applications et Avantages du Web Scraping

Réalisation d'Études de Marché et d'Analyses Consommateur

- Les entreprises peuvent collecter des données sur les comportements et préférences des consommateurs, pour comprendre les tendances et personnaliser les offres.
- Cette technique permet de suivre les avis et commentaires clients, offrant ainsi des informations précieuses sur la perception de produits et services.

Prise de Décisions Stratégique et Optimisation des Ressources

- En réunissant des données issues de différentes sources, le Web scraping fournit aux décideurs une vue d'ensemble structurée et riche.
- Les informations ainsi collectées permettent d'identifier des opportunités stratégiques, de mieux comprendre le marché et de réduire les risques liés à la prise de décision.

❑ Légalité et Éthique du Web Scraping

Est-ce que le Web Scraping est légal ?

- Le Web scraping est légal dès lors qu'il s'agit de collecter des informations publiques et que les données sont utilisées à des fins personnelles ou non commerciales.
- Cependant, des restrictions s'appliquent à l'utilisation des données, en particulier dans un contexte commercial, et certaines pratiques peuvent enfreindre les droits d'auteur et la vie privée.

❑ Légalité et Éthique du Web Scraping

Bonnes Pratiques pour Assurer la Conformité Légale et Éthique

- Respectez les droits d'auteur et les conditions d'utilisation : Chaque site web peut avoir des règles spécifiques sur l'accès et la réutilisation de ses données.
- Évitez de contourner les protections anti-scraping : Certains sites utilisent des techniques comme les CAPTCHA ou des limitations de requêtes pour restreindre l'accès automatisé. Contourner ces mesures pourrait être considéré comme une violation.
- Utilisez les données de manière responsable et transparente : Assurez-vous de ne collecter que les informations nécessaires et de ne pas surcharger les serveurs des sites.

❑ Manipulation des Pages HTML avec urllib.request

Introduction

- Pour interagir avec des pages web en Python, urllib.request est un module utile pour ouvrir et lire le contenu HTML des pages web.

Étape 1 : Comprendre le module urllib.request

- Le module urllib.request permet d'envoyer des requêtes HTTP pour accéder à des ressources web. La méthode principale utilisée ici est urlopen, qui ouvre une connexion à une page web.

Étape 2 : Importation de urlopen

- Avant de commencer, il faut importer la méthode urlopen du module urllib.request.

Exemple : from urllib.request import urlopen

❑ Manipulation des Pages HTML avec urllib.request

Étape 3 : Ouverture d'une page web

- Pour ouvrir une page web, on utilise l'URL de la page en tant que paramètre de la méthode urlopen. Cette méthode retourne un objet de type http.client.HTTPResponse.

Exemple :

```
url = "https://example.com/page.html"
page = urlopen(url)
print(page)  # Affiche l'objet HTTPResponse
```

Étape 4 : Lecture et décodage du contenu HTML

- L'objet HTTPResponse contient les données brutes de la réponse. Pour obtenir le contenu HTML, on utilise les méthodes .read() pour lire les données et .decode("utf-8") pour les convertir en une chaîne de caractères lisible.

❑ Manipulation des Pages HTML avec BeautifulSoupavec urllib.request

Exemple :

```
url = "https://example.com/page.html"
page = urlopen(url)
# Lecture et décodage
web_page = page.read().decode("utf-8")
print(type(web_page))  # Vérifie que le contenu est une chaîne (str)
print(web_page)  # Affiche le contenu HTML
```

Remarque sur le décodage

Le décodage en UTF-8 est essentiel car il permet de convertir les données binaires en une chaîne de caractères lisible. L'encodage UTF-8 est largement utilisé sur le web, ce qui en fait un choix par défaut.

Explications

1. **urlopen(url)** : Envoie une requête à l'URL spécifiée et retourne une réponse HTTP.
2. **.read()** : Lit les données brutes de la réponse.
3. **.decode("utf-8")** : Convertit les données en une chaîne lisible en utilisant l'encodage UTF-8.
4. **type()** : Permet de vérifier le type de données obtenu.

❑ Analyse d'une page HTML pour compter les éléments spécifiques

Cette partie montre comment travailler avec le contenu HTML d'une page web une fois qu'il a été chargé et converti en une chaîne de caractères. On va se concentrer sur l'analyse des balises HTML spécifiques, comme et <img>.

Étape 1 : Importation et lecture de la page HTML

Le code commence par importer le module urllib.request pour accéder à la page web, la lire et la décoder en UTF-8 pour obtenir une chaîne lisible.

Exemple de code source:

```
from urllib.request import urlopen
# Ouverture de la page web
url = " https://example.com/page.html "
page = urlopen(url)
# Lecture et décodage
web_page = page.read().decode("utf-8")
```

❑ Analyse d'une page HTML pour compter les éléments spécifiques

Étape 2 : Utilisation des méthodes de chaînes pour localiser les éléments HTML

Une fois que la page est lue, le contenu est une chaîne de caractères. On peut appliquer des méthodes de chaînes pour analyser les balises HTML. La méthode .count() est particulièrement utile pour compter les occurrences des éléments spécifiques.

1. **Compter les balises **

- Comme les balises peuvent contenir des attributs (ex. :), on cherche la chaîne '<div' pour inclure toutes les variations.

Exemple :

```
div_count = web_page.count('<div')
print(f"Nombre de balises <div> : {div_count}")
```

❑ Analyse d'une page HTML pour compter les éléments spécifiques

Étape 2 : Utilisation des méthodes de chaînes pour localiser les éléments HTML

2. Compter les balises <img>

- Les images sont définies avec des balises <img>, qui peuvent également contenir des attributs (ex. : <img src="image.jpg" alt="Example">).

Exemple :

```
img_count = web_page.count('<img')
print(f"Nombre de balises <img> : {img_count}")
```

❑ Analyse d'une page HTML pour compter les éléments spécifiques

Tâche

Pour la page **https://example.com/page.html**, réaliser les taches suivantes :

1. Compter le nombre de balises présentes dans web_page.
2. Compter le nombre d'images (balises <img>) présentes dans web_page.

```
from urllib.request import urlopen
# Ouverture et lecture de la page web
url = " https://example.com/page.html"
page = urlopen(url)
web_page = page.read().decode("utf-8")
div_count = web_page.count('<div')                # Compter les balises <div>
print(f"Nombre de balises <div> : {div_count}")
img_count = web_page.count('<img')                # Compter les balises <img>
print(f"Nombre de balises <img> : {img_count}")
```

❑ Analyse et manipulation des pages HTML avec BeautifulSoup

Installation et importation de BeautifulSoup

Pour utiliser cette bibliothèque, il faut l'installer en exécutant la commande suivante dans le terminal ou l'invite de commande :

pip install beautifulsoup4

Ensuite, l'importation de BeautifulSoup se fait de cette manière :

from bs4 import BeautifulSoup

BeautifulSoup est spécifiquement conçu pour analyser les fichiers HTML et ne gère pas directement les liens. Pour cela, l'utilisation de urlopen du module urllib.request est nécessaire.

❑ Analyse et manipulation des pages HTML avec BeautifulSoup

Création d'un objet BeautifulSoup

Pour commencer l'analyse, il faut fournir deux paramètres à la fonction BeautifulSoup :

1. Le contenu HTML (sous forme de chaîne de caractères).
2. Le parseur (ici, html.parser, qui est intégré à Python).

Exemple : Lecture et analyse d'une page web

```
from bs4 import BeautifulSoup
from urllib.request import urlopen
url = "https://example.com/page.html"  # Remplacer par l'URL cible  # Lecture de la page web
page = urlopen(url)
html = page.read().decode("utf-8")
soup = BeautifulSoup(html, 'html.parser')       # Création de l'objet BeautifulSoup
print(type(soup))  # Vérifie le type de l'objet créé
print(soup)  # Affiche la structure HTML brute
```

❑ Analyse et manipulation des pages HTML avec BeautifulSoup

Méthode .prettify() : Présentation structurée du HTML

La méthode .prettify() de BeautifulSoup permet de représenter le contenu HTML dans une structure hiérarchique, facilitant sa lecture.

Exemple :

```
print(soup.prettify())
```

Navigation dans les éléments HTML

Avec BeautifulSoup, chaque balise HTML devient un attribut accessible. Par exemple, on peut accéder directement à la balise <head> ou <body>.

Exemple : Analyse de l'élément <head>

```
print(soup.head.prettify())
```

On peut également explorer d'autres balises en remplaçant .head par .body ou tout autre élément.

❑ Analyse et manipulation des pages HTML avec BeautifulSoup

Exemple :

```
from bs4 import BeautifulSoup

# Contenu HTML de la page

html = "<html><head><title>Ma Super Page</title><meta charset="UTF 8"></head><body><h1> Bienvenue !</h1>  <p> Ceci est un paragraphe.</p></body></html>"

# Créer l'objet BeautifulSoup

soup = BeautifulSoup(html, 'html.parser')

print(soup.head.prettify())
```

```
<head>
 <title>
  Ma Super Page
 </title>
 <meta charset="UTF-8"/>
</head>
```

❑ Analyse et manipulation des pages HTML avec BeautifulSoup

Parcours des éléments enfants

Pour explorer les enfants d'une balise, la propriété .children permet d'itérer à travers eux avec une boucle for.

Pour parcourir tous les éléments contenus dans une balise, à tous les niveaux, et pas seulement ses enfants directs, on utilise .descendants.

Exemple : Parcours des enfants de <head>

```
for child in soup.head.children:
    print(child)
```

Exemple : parcourt tous les descendants, y compris les enfants, petits-enfants de <head>

```
for desc in soup.head.descendants:
    print(desc)
```

❑ Analyse et manipulation des pages HTML avec BeautifulSoup

Exemple : Accès au premier <h1> de la page

Pour récupérer le contenu de la première balise <h1> :

```
from bs4 import BeautifulSoup
from urllib.request import urlopen
# Lecture de la page web
url = "https://example.com/page.html"
page = urlopen(url)
html = page.read().decode("utf-8")
# Analyse HTML
soup = BeautifulSoup(html, 'html.parser')
# Extraction et affichage du premier <h1>
first_h1 = soup.h1
print(first_h1.text)
```

```
<html>
<body>
  <h1>Bienvenue</h1>
  <p>Bonjour les
étudiants</p>
  <h1>Deuxième
titre</h1>
</body>
</html>
```

Bienvenue

❑ Exploration des éléments HTML avec BeautifulSoup

Lors de la navigation dans un document HTML avec BeautifulSoup, accéder directement à un élément via un attribut renvoie uniquement la première occurrence. Cependant, pour trouver un élément particulier sans connaître son chemin exact, la méthode .find() peut être utilisée. Elle permet de rechercher la première occurrence d'une balise donnée, en spécifiant son nom sous forme de chaîne de caractères.

❑ Exploration des éléments HTML avec BeautifulSoup

Recherche de la première occurrence d'un élément avec .find()

La méthode **.find()** localise la première balise spécifiée dans le document HTML. Elle est utile lorsque l'on veut récupérer rapidement un élément particulier.

*Exemple : Localisation de la première balise *

```
from bs4 import BeautifulSoup
from urllib.request import urlopen
url = "https://example.com/page.html"
# Lecture de la page web
page = urlopen(url)
html = page.read().decode("utf-8")
# Analyse HTML avec BeautifulSoup
soup = BeautifulSoup(html, 'html.parser')
# Recherche de la première balise <div>
print(soup.find('div'))
```

❑ Exploration des éléments HTML avec BeautifulSoup

Recherche de toutes les occurrences d'un élément avec .find_all()

Pour obtenir toutes les occurrences d'une balise, la méthode .find_all() peut être utilisée. Cette méthode renvoie une liste contenant toutes les instances de la balise spécifiée.

Exemple : Localisation de toutes les balises <p>

```
# Recherche de toutes les balises <p>
print(soup.find_all('p'))
```

Recherche de plusieurs balises simultanément

La méthode .find_all() accepte également une liste de balises, permettant de rechercher plusieurs types d'éléments en une seule commande.

Exemple : Localisation des balises et <title>

```
for el in soup.find_all(["div", "title"]):
    print(el)
```

❑ Extraction des attributs et du contenu des balises HTML avec BeautifulSoup

BeautifulSoup permet d'accéder non seulement aux balises spécifiques d'un document HTML, mais aussi à leurs attributs et à leur contenu. Ces fonctionnalités sont essentielles pour analyser et extraire des données de manière ciblée.

Récupération des attributs d'une balise avec .attrs

L'attribut .attrs retourne un dictionnaire contenant les attributs d'une balise HTML. Les clés du dictionnaire sont les noms des attributs et les valeurs correspondent à leur contenu.

```
from bs4 import BeautifulSoup
html = "<img src="photo.jpg" alt="Une photo" class="image">"
soup = BeautifulSoup(html, "html.parser")
# Accéder aux attributs
print(soup.img.attrs)
```

```
{'src': 'photo.jpg', 'alt': 'Une photo', 'class': ['image']}
```

❑ Extraction des attributs et du contenu des balises HTML avec BeautifulSoup

Accès au contenu d'une balise avec .contents

L'attribut .contents retourne une liste contenant tous les éléments enfants d'une balise HTML. Cela inclut les caractères de nouvelle ligne (\n), qui peuvent alourdir les résultats.

```
from bs4 import BeautifulSoup
html = "<div>Bonjour<p>Paragraphe 1</p><p>Paragraphe 2</p></div>"
soup = BeautifulSoup(html, "html.parser")
# Accéder au contenu (liste des enfants directs)
print(soup.div.contents)
```

['Bonjour', Paragraphe 1 , Paragraphe 2]

❑ Extraction des attributs et du contenu des balises HTML avec BeautifulSoup

*Exemple : Extraction des attributs de la première balise *

```
from bs4 import BeautifulSoup
from urllib.request import urlopen
url = "https://example.com/page.html"
page = urlopen(url)
html = page.read().decode("utf-8")
# Analyse HTML avec BeautifulSoup
soup = BeautifulSoup(html, 'html.parser')
# Récupération des attributs de la première balise <div>
print(soup.find('div').attrs)

Exemple : Extraction du contenu de la première balise <div>
print(soup.find('div').contents)
```

❑ Extraction des attributs et du contenu des balises HTML avec BeautifulSoup

Extraction du texte avec .get_text()

Pour extraire uniquement le texte contenu dans une balise, sans inclure les caractères de nouvelle ligne ni les balises enfants, la méthode .get_text() est utilisée. Elle retourne une chaîne de caractères propre.

***Exemple : Récupération du texte dans la première balise ***

```
print(soup.find('div').get_text())
```

Comparaison des méthodes

.contents : Retourne une liste de tous les éléments enfants (inclut les caractères de nouvelle ligne et les balises enfants).

.get_text() : Retourne uniquement le texte contenu dans la balise, sans balises ni caractères inutiles.

❑ Extraction des attributs et du contenu des balises HTML avec BeautifulSoup

Tâche : Extraction des attributs et du texte

Objectif

- Extraire les attributs de la première balise <p>.
- Récupérer uniquement le texte de la première balise <ul>.

❑ Extraction des attributs et du contenu des balises HTML avec BeautifulSoup

Solution

```
from bs4 import BeautifulSoup
from urllib.request import urlopen
url = "https://example.com/page.html"
page = urlopen(url)
html = page.read().decode("utf-8")
soup = BeautifulSoup(html, 'html.parser')    # Analyse HTML avec BeautifulSoup
p_attributes = soup.find('p').attrs      # Attributs de la première balise <p>
print(f"Attributs de la première balise <p> : {p_attributes}")
# Texte de la première balise <ul>
ul_text = soup.find('ul').get_text()
print(f"Texte de la première balise <ul> : {ul_text}")
```

❑ filtrage avec BeautifulSoup

Introduction

Toutes les méthodes abordées précédemment, comme .find() et .get_text(), peuvent également être appliquées à toutes les balises retournées par la méthode .find_all(). Cependant, comme .find_all() retourne une liste d'éléments, chaque élément doit être traité individuellement en utilisant une boucle for.

Extraction des attributs de plusieurs balises

Pour récupérer les attributs de plusieurs balises, on peut utiliser .attrs sur chaque élément de la liste retournée par .find_all(). Cela est utile pour analyser des balises avec des attributs obligatoires, comme <a> (avec href) ou <img> (avec src).

❑ filtrage avec BeautifulSoup

Extraction du texte de plusieurs balises

La méthode .get_text() peut également être utilisée dans une boucle pour extraire le texte de toutes les balises trouvées par .find_all().

*Exemple : Récupération des attributs de toutes les balises *

```
from bs4 import BeautifulSoup
from urllib.request import urlopen
url = "https://example.com/page.html"
page = urlopen(url)
html = page.read().decode("utf-8")
# Analyse HTML avec BeautifulSoup
soup = BeautifulSoup(html, 'html.parser')
# Extraction des attributs de toutes les balises <div>
for div in soup.find_all('div'):
    print(div.attrs)
```

Exemple : Extraction du texte de toutes les balises <p>

```
for p in soup.find_all('p'):
    print(p.get_text())
```

❑ filtrage avec BeautifulSoup

Extraction du texte de plusieurs balises

La méthode .get_text() peut également être utilisée dans une boucle pour extraire le texte de toutes les balises trouvées par .find_all().

Exemple : Extraction du texte de toutes les balises <p>

```
for p in soup.find_all('p'):
    print(p.get_text())
```

Extraction d'attributs spécifiques avec .get()

Pour cibler un attribut spécifique (comme href ou src), .get() est utilisé après .attrs.

Exemple : Récupération des attributs src de toutes les balises <img>

```
for img in soup.find_all('img'):
    print(img.attrs.get('src'))
```

❑ filtrage avec BeautifulSoup

Filtrage des balises avec des attributs spécifiques

Pour trouver des balises avec des attributs spécifiques, comme une classe particulière ou un identifiant (id), il est possible d'utiliser un dictionnaire dans la méthode .find_all().

Exemple : Filtrage des balises avec une classe box

```
for div in soup.find_all("div", {"class": "box"}):
    print(div)
```

Exemple : Recherche d'une balise <p> avec un id spécifique

```
print(soup.find("p", {"id": "id2"}))
```

Extraction des classes des balises

Pour afficher les classes d'une balise, on peut récupérer la valeur de l'attribut class avec .attrs.get().

***Exemple : Extraction des classes des balises ***

```
for div in soup.find_all('div'):
    print(div.attrs.get('class'))
```